Migration Wave Orchestration, Throttling Management & Post-Migration Validation

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

1. Solution Overview — Migration Wave Orchestration, Throttling Management & Post-Migration Validation 4

What this solution is . . . 4

How it relates to the other migration solutions (positioning) . . . 5

How it works — the cutover-operations model . . . 5

Reference model — one wave, end to end . . . 6

Benefits . . . 6

Limitations and boundaries . . . 7

Deployment model . . . 7

2. Requirements 8

2.1 A chosen, connected migration engine . . . 8

2.2 A discovery user set (the population to batch) . . . 8

2.3 Documented throttling / API-limit model for both vendors . . . 9

2.4 Service identity for the automation (read-only / least privilege) . . . 9

2.5 Defined validation criteria (the pass/fail definition of "done") . . . 10

2.6 A hypercare team, SLA, and issue intake . . . 10

2.7 A change window and rollback authority for cutover . . . 10

Global Requirements — Google Workspace & Gemini Solutions 10

1. Google Workspace / Cloud Identity tenant . . . 11

2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 11

3. DNS control for the routed domains . . . 11

4. Identity & provisioning source of truth . . . 12

5. A pilot scope (OU / group) and change control . . . 12

6. Security, compliance & governance stakeholders . . . 12

3. Implementation Guide 13

Stage A — Assess (profile the population and the limits) . . . 13

Stage B — Design (batch the waves, budget the throttle) . . . 13

Stage C — Configure (apply the budget, stage the artifacts) . . . 14

Stage D — Pilot (run the pilot wave end to end: pre-stage → delta → cutover) . . . 15

Stage E — Validate (prove the pilot wave is done) . . . 15

Stage F — Expand (roll the proven runbook wave by wave) . . . 16

Artifact register (named, reused everywhere) . . . 16

5. Testing & Validation 17

T1 — Discovery profile completeness . . . 17

T2 — Cluster integrity in the wave plan . . . 17
T3 — Throttle budget fits the window . . . 17
T4 — Pre-stage throttling stays in budget . . . 18
T5 — Cutover readiness (go/no-go) . . . 18
T6 — Item-count parity (post-migration) . . . 18
T7 — Permission & sharing parity . . . 18
T8 — Mail flow & delegates . . . 18
T9 — Calendar & free/busy . . . 19
T10 — Hypercare SLA compliance . . . 19
T11 — Safe-by-default tool behavior . . . 19
6. Troubleshooting 19
Throttling & API limits . . . 19
Wave design / clusters . . . 20
Item counts / delta completeness . . . 20
Permissions & sharing . . . 21
Mail flow & coexistence . . . 21
Calendar / free-busy . . . 22
Automation & credentials . . . 22
Hypercare . . . 22
7. Best Practices 23
Wave design . . . 23
Throttling & API limits . . . 23
Sequencing (pre-stage → delta → cutover) . . . 23
Post-migration validation . . . 24
Hypercare . . . 24
Automation & safety . . . 24
8. Deliverables Summary 24
Documentation . . . 25
The 5 Python tools (`scripts/`) . . . 25
Appendix A — PowerShell Scripts 27

Executive Summary

Outcome. Enterprise Microsoft 365 → Google Workspace migrations succeed or fail in the **cutover**, not the plan. This solution delivers the **tool-agnostic cutover-operations layer** that turns a migration plan into clean, evidenced execution: users batched into dependency-aware **waves**, concurrency **throttle-budgeted** against both Microsoft Graph and Google API limits, each wave **sequenced** pre-stage → delta → cutover through explicit go/no-go gates, then **validated** after the move (item counts, permissions, mail flow, delegates, calendar) and **hypercare-triaged** against an SLA. The result is cutovers that finish clean, with evidence — not surprises.

Approach. The solution sits *over* whatever migration engine the organization has chosen (Google Workspace Migrate, GWMME, BitTitan MigrationWiz, CloudM Migrate, or another) — it orchestrates and validates around the engine rather than replacing it, so the same discipline applies regardless of tool. It follows a **pilot-first** model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — proving the full runbook on a small, representative pilot wave before rolling it out wave by wave. Five Python tools (Google Workspace Admin SDK / API automation, with Microsoft Graph read where needed) handle discovery, wave batching, throttle budgeting, cutover-readiness + validation reporting, and hypercare triage. All are **read-only or dry-run by default**, use least-privilege OAuth/ADC, and carry no secrets in code.

Why it's low-risk. Every high-impact change — mail routing, DNS, cutover — is made under change control with an explicit pass/fail gate and a documented, no-data-loss **rollback** (reverting routing returns a wave to coexistence). Nothing rolls out until a **passing pilot** proves it. Concurrency is budgeted against both vendors' real limits *before* a wave runs, so throttling is managed rather than discovered. And "done" is defined as an **evidenced Validation Report plus a clear hypercare SLA**, so each wave is provably complete before the next begins. It is engine-agnostic, self-guided, tested, and delivered at a fraction of the cost of bespoke cutover consulting.

1. Solution Overview — Migration Wave Orchestration, Throttling Management & Post-Migration Validation

What this solution is

A migration **plan** is not a migration **execution**. This solution is the **tool-agnostic cutover operations layer** that sits over whatever migration engine the organization has chosen — Google Workspace Migrate, Google Workspace Migration for Microsoft Exchange (GWMME), BitTitan MigrationWiz, CloudM Migrate, or another. It does not move data itself; it **orchestrates** the people-and-timing work around the engine so that cutovers finish clean, with evidence:

- **Wave batching** — turn a flat user list from Microsoft 365 into ordered, sized, dependency-aware migration waves (keeping delegates, shared mailboxes, and team members together).
- **Throttling & API-limit management** — plan concurrency and pacing against both the **Microsoft Graph **throttling model and the** Google Workspace Admin SDK / API** quotas, so waves finish inside their windows without tripping 429/`RESOURCE_EXHAUSTED` throttling on either side.
- **Pre-stage → delta → cutover sequencing** — run the standard three-pass pattern (bulk pre-stage of

historical data, one or more incremental deltas, then a final delta + cutover at the change window) with explicit go/no-go gates.

• **Post-migration validation** — after each wave, verify **item counts, folder/label structure, permissions and sharing, mail flow, delegates, and calendar/free-busy** against the source, and produce an evidence report per user and per wave.

• **Hypercare issue triage** — track, categorize, and SLA the issues raised in the days after each cutover, so the tail of a migration is managed, not fire-fought.

How it relates to the other migration solutions (positioning)

This solution is deliberately scoped to **cutover operations and validation**, and it complements — does not duplicate — the other migration solutions in the catalog:

• The **program/plan solution** (M365 → Google Workspace Migration Program & Wave Plan) owns the **program-level strategy**: discovery baseline, the overall wave *plan*, tooling strategy, coexistence design, and the multi-month operating model. It decides *what* the waves are and *why*.

• The **per-service migration solutions** (Exchange Online → Gmail, SharePoint/OneDrive → Google Drive, Teams → Google Chat/Meet) and the **tool-specific solutions** (Google Workspace Migrate, GWMME, BitTitan, CloudM) own the **data-movement mechanics** — the engine that actually copies mail, files, and chats.

• **This solution** is the hands-on **execution and evidence toolkit** that runs *between* them: it takes the plan's waves and the engine's capabilities and turns them into a repeatable cutover runbook — batch, throttle-budget, sequence, validate, triage — that any engine can be driven by. It is **engine-agnostic on purpose**: the same wave, throttle, validation, and hypercare discipline applies whether the bytes are moving through Google's tool or a third party's.

If you are running only one small wave with one engine, the tool's own docs may be enough. This solution earns its place at **enterprise scale** (tens of thousands of users, many waves, two vendors' rate limits, and an auditor who wants proof the data arrived).

How it works — the cutover-operations model

The solution is organized around five operating capabilities, each backed by a Python tool and a named artifact convention (`<Purpose> – <Scope> – <Stage>`, e.g. `Migration Wave – Finance – Pilot`):

1. **Wave design.** From the discovery user set (exported from Microsoft 365 / Entra ID), build waves that (a) fit a target size per wave, (b) keep **dependency clusters together** — a manager and their delegates, a shared mailbox and its members, a department that shares Drive content — and (c) sequence a **pilot wave first**. Each wave is a named artifact (`Migration Wave – <Dept> – <Stage>`) reused everywhere downstream.

1. **Throttling / API-limit budgeting.** For each wave, compute a **throttle budget**: how many concurrent migration workers and what pacing keep you under the **Microsoft Graph** per-app/ per-tenant throttling limits (read side) and the **Google Admin SDK / Drive / Gmail API** quotas (write side), plus the

practical throughput of the chosen engine. The budget is the input to the engine's concurrency settings and to the wave schedule. (Exact current limits differ by API and by tenant/licensing — the tool reads Google quota where the API exposes it and applies documented Microsoft Graph throttling guidance; **verify current numbers for your tenant and engine version**.)

1. **Sequencing (pre-stage → delta → cutover).** Each wave runs the three-pass pattern with gates:

pre-stage (bulk historical copy, no user impact, runs days ahead), **delta** (incremental catch-up passes), then **cutover** (final delta + mail-routing/DNS switch at the change window). Each transition has a documented **pass/fail gate** and a **rollback**.

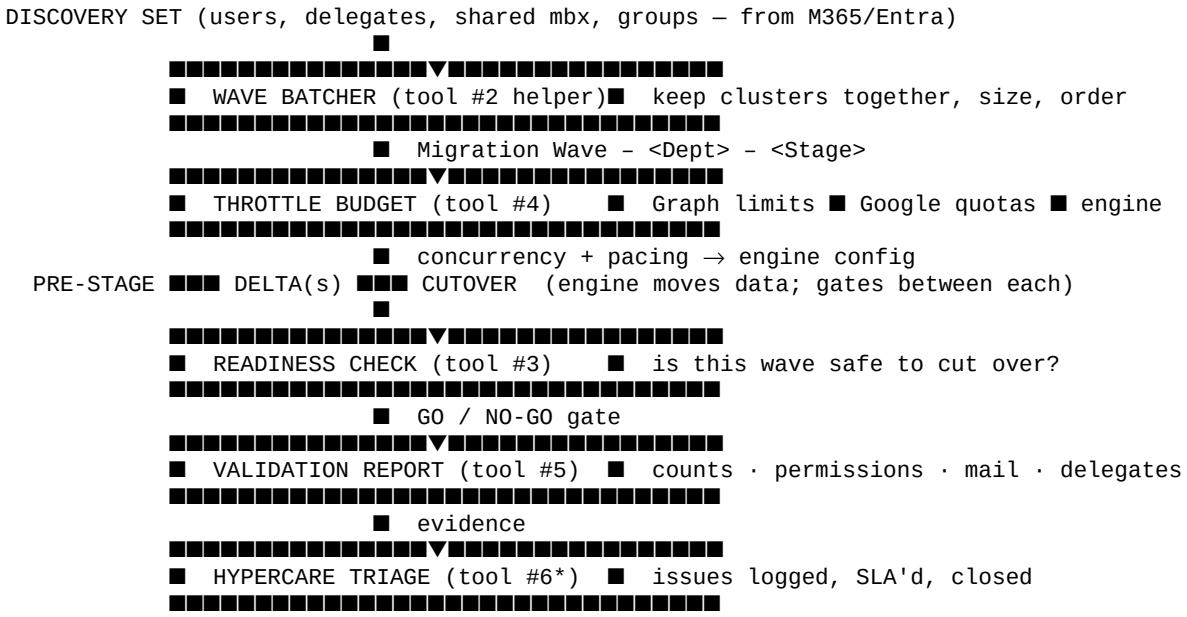
1. **Post-migration validation.** After cutover, validate the wave against the source: item counts by

folder/label, permission/sharing parity, mail-flow (test messages route correctly and old address still resolves during coexistence), delegate access, and calendar/free-busy. Output is a per-wave **Validation Report** artifact — the evidence the wave is genuinely done.

1. **Hypercare triage.** For the hypercare window after each cutover, log every user-reported issue

into a triage tracker (category, severity, SLA, owner), so the post-cutover tail is measured and closed against an SLA rather than absorbed as noise.

Reference model — one wave, end to end



(The five Python tools are: assessment/discovery, wave batcher, cutover-readiness checker, throttle/API-limit budget planner, and the validation-report + hypercare-triage helpers — see the Deliverables Summary for the exact five.)

Benefits

• **Cutovers finish clean, with evidence.** Every wave produces a Validation Report — item counts,

permissions, mail flow, delegates, calendar — so "done" is proven, not asserted.

• **No throttling surprises.** Concurrency and pacing are budgeted against both vendors' real limits

before a wave starts, not discovered mid-cutover when a 429 storm stalls the move.

- **Repeatable.** The pilot wave proves the runbook; later waves execute the same batch → throttle → sequence → validate → triage pattern, so the team stops re-learning the same lessons.
- **Engine-agnostic.** The discipline outlives any one tool — swap engines and the wave, throttle, validation, and hypercare layers still apply.
- **Auditable.** Named artifacts and per-wave evidence give security/compliance and the business a clean trail that data arrived and access was preserved.

Limitations and boundaries

- **This is not a migration engine.** It does not copy mail, files, or chats. It orchestrates and validates around the engine the organization has chosen; the actual data movement is the engine's job (and its own solution/docs).
- **Tool internals are deferred to the vendor.** Deep console screens, exact throughput per SKU, licensing, and support for Google Workspace Migrate / GWMME / BitTitan / CloudM live in each vendor's own documentation. This solution drives them at the **playbook level** (mappings, CSVs, wave schedules, validation) and via their documented API/CSV interfaces where one exists — it does **not** invent private console paths or version-specific behavior. Where behavior is version- or preview-dependent, the guidance says **"verify for your version."**
- **Current rate limits change.** Microsoft Graph throttling and Google API quotas are adjusted by the vendors over time and vary by license/tenant. The throttle planner applies documented models and reads live Google quota where exposed, but the operator must **confirm current numbers** for their tenant and engine before committing a wave schedule.
- **Coexistence and DNS are high-impact.** Mail-routing and DNS cutover changes affect production mail flow; they are made under change control with the rollback documented in the Implementation Guide.

Deployment model

This solution follows the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — **pilot-first**, with explicit pass/fail gates and a documented rollback for every high-impact change (mail routing, DNS, cutover):

- **Assess** — export and profile the discovery user set; confirm engine, admin access on both sides, and validation criteria.
- **Design** — batch the waves, choose sizes/order, and compute the throttle budget per wave.
- **Configure** — apply the throttle budget to the engine, stage the wave's mappings, and prepare the validation and triage artifacts.
- **Pilot** — run the **pilot wave** (pre-stage → delta → cutover) end to end; prove the runbook.
- **Validate** — run the Validation Report and clear the gate; only then expand.

• **Expand** — roll the proven runbook wave-by-wave across the estate, triaging hypercare each time.

2. Requirements

These are the **solution-specific** prerequisites for running migration cutover operations. They build **on top of** the shared category prerequisites in `global-requirements-google.md` (Workspace/Cloud Identity tenant with super-admin; source Microsoft 365 / Entra access; DNS control for routed domains; identity source of truth and provisioning; a pilot OU/group with change control; security/compliance stakeholders). Do not re-satisfy those here — verify them first, then confirm the additions below.

> *This solution assumes the **program-level plan already exists** (the migration program solution owns discovery baseline, tooling strategy, and coexistence design). It also assumes a **migration engine has been chosen and stood up** (its own solution/docs own the engine). What this solution adds is the operational scaffolding to *run* the waves: the user set to batch, the limits to budget against, the validation criteria, and the hypercare team.*

2.1 A chosen, connected migration engine

• **Description:** One migration engine is selected and its source/target connections are established:

Google Workspace Migrate, **GWMME**, **BitTitan MigrationWiz**, **CloudM Migrate**, or another. The engine can authenticate to the **Microsoft 365 source** (Exchange Online / OneDrive / SharePoint) and the **Google Workspace target** at the scope it will migrate.

• **Why required:** This solution orchestrates *around* an engine; it does not move data. Wave scheduling, throttle budgeting, and validation all assume the engine exists and can run.

• **How to verify:** Confirm the engine is installed/subscribed, its source and target endpoints authenticate successfully, and a single test user can be migrated end to end. (Engine console details are in that vendor's own documentation — **verify for your version**.)

• **How to implement if missing:** Stand up the engine per its vendor solution/documentation and validate one test user before batching waves.

2.2 A discovery user set (the population to batch)

• **Description:** A machine-readable export of the users in scope, with the attributes needed to batch waves: user principal name / primary SMTP, display name, department/OU, manager, **mailbox delegates**, **shared-mailbox membership**, distribution-list/group membership, and (where known) mailbox and Drive/OneDrive data volume.

• **Why required:** Waves must keep **dependency clusters** together (a manager and their delegates, a shared mailbox and its members) and be sized by data volume; without these attributes the batcher can only split alphabetically, which breaks delegate and shared-mailbox access at cutover.

• **How to verify:** Open the export (CSV/JSON) and confirm each user row carries UPN, department,

manager, and delegate/shared-mailbox references. The assessment tool (`discovery-profile.py`) reports which attributes are missing.

- **How to implement if missing:** Produce the export from Microsoft Entra ID / Exchange Online (the

program solution's discovery step) — or use the assessment tool to read the Google directory for the target-side view — before batching.

2.3 Documented throttling / API-limit model for both vendors

- **Description:** The **current** throttling/quota posture on both sides: the **Microsoft Graph**

throttling model for the read APIs the engine uses (per-app and per-tenant limits, 429 + `Retry-After` behavior), and the **Google Workspace** API quotas relevant to the target and to your automation (Admin SDK Directory, Reports, Drive, Gmail), plus the engine's own documented throughput/concurrency guidance.

- **Why required:** The throttle budget (concurrency + pacing per wave) is computed from these. Guessing

causes either wasted windows (too conservative) or 429/`RESOURCE_EXHAUSTED` storms that stall a cutover (too aggressive).

- **How to verify:** In **Google Cloud console** → **APIs & Services** → **Quotas** confirm the quotas for the

APIs your automation and engine use; review Microsoft Graph throttling guidance for the resources the engine reads; review the engine's concurrency guidance. **These change over time and vary by tenant/licensing — confirm current numbers.**

- **How to implement if missing:** Gather the current limits (the throttle planner reads Google quota

where the API exposes it and applies documented Microsoft Graph guidance); request quota increases from the respective vendor where a wave needs more than the default.

2.4 Service identity for the automation (read-only / least privilege)

- **Description:** A credential for the Python tools to read both directories and (optionally) validate

the target: a **Google Cloud service account with domain-wide delegation** (or an OAuth client) scoped to the **read-only** Admin SDK Directory / Reports / Drive / Gmail scopes needed for validation, and — where the Microsoft side must be read — a **Microsoft Entra app registration** with least-privilege Graph read scopes.

- **Why required:** The tools discover users, budget throttling, check readiness, and validate results;

they need directory/report read access. Least privilege keeps the automation safe.

- **How to verify:** Confirm the Google service account is authorized in **Admin console → Security →

API controls → Domain-wide delegation** with only the read scopes listed in each tool's `--help`; if reading Microsoft, confirm the Entra app has only the read Graph scopes.

- **How to implement if missing:** Create the service account / app registration, grant the minimal

read-only scopes, and provide credentials to the tools via **OAuth / Application Default Credentials (ADC)** — **never** hardcode keys or commit exported key files.

2.5 Defined validation criteria (the pass/fail definition of "done")

• **Description:** An agreed, written definition of what "successfully migrated" means for a user and a wave: acceptable item-count variance (some engines legitimately deduplicate or skip system items), which permission/sharing states must match, mail-flow tests to pass, delegate access to confirm, and calendar/free-busy checks.

• **Why required:** Validation is only meaningful against a criterion. Without it, "counts differ by 3" is either a pass or a failure depending on nobody's opinion.

• **How to verify:** Confirm a validation-criteria document exists and names the tolerances; the validation tool takes these as parameters/config.

• **How to implement if missing:** Agree the criteria with the migration lead and the business owners before the pilot wave; record them so every wave is judged the same way.

2.6 A hypercare team, SLA, and issue intake

• **Description:** A named **hypercare team** for the post-cutover window, an agreed **SLA** per issue severity, and an intake channel (ticket queue / form) that feeds the triage tracker.

• **Why required:** Cutovers generate a tail of user issues (a missing delegate, a broken mail rule, a not-yet-shared Drive). Managing that tail against an SLA is what keeps a wave from being declared "done" while users are still blocked.

• **How to verify:** Confirm the team roster, the severity/SLA matrix, and the intake channel exist.

• **How to implement if missing:** Stand up the hypercare rota and SLA and connect the intake channel before the pilot wave cuts over.

2.7 A change window and rollback authority for cutover

• **Description:** An approved **change window** for each cutover (final delta + mail-routing/DNS switch) and a named approver who can authorize the **rollback** if a gate fails.

• **Why required:** Cutover changes mail flow and DNS — high-impact, production-affecting changes that must be made under change control with a way back.

• **How to verify:** Confirm the change window is booked and the rollback approver is named for each wave (especially the pilot).

• **How to implement if missing:** Book the window and confirm the rollback approver as part of the per-wave go/no-go, per the shared change-control requirement.

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The* ***Google Admin console*** *(admin.google.com),* ***Google Cloud / Cloud Identity****, and the* ***Microsoft / Entra*** *sides are authored at exact, current click-by-click detail from the vendors' public documentation.* ***Gemini Enterprise*** *capabilities that are in preview or rolling out are* ***cited or flagged "verify in your tenant"*** *— never invented.* ***Third-party migration tools*** *(e.g. for mail/Drive/Chat data movement) and* ***third-party enterprise apps*** *(ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

• **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).

• **Why required:** All configuration, policy, and API work is performed against this tenant.

• **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.

• **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

• **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model; the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the owners before making production changes.

6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance** stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

3. Implementation Guide

This guide runs the cutover-operations layer end to end following the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — **pilot-first**, with explicit pass/fail gates and a documented rollback for every high-impact change. Every artifact is named with the convention `<Purpose> – <Scope> – <Stage>` and that exact name is reused in testing, troubleshooting, and the scripts.

Prerequisites: complete Section 2 (chosen/connected engine, discovery user set, both-vendor limit model, read-only service identity, validation criteria, hypercare team + SLA, change window + rollback authority) and the shared `global-requirements-google.md`.

> ***Engine boundary.*** *Where a step says "configure the engine's concurrency" or "start the pre-stage pass," perform that action in your chosen engine (Google Workspace Migrate / GWMME / BitTitan / CloudM)* ***per that vendor's own documentation*** *— the exact console labels and throughput are the vendor's, and are* ***version-dependent (verify for your version)****. This guide owns the *orchestration* around those actions, not the engine internals.*

Stage A — Assess (profile the population and the limits)

A1. Export and profile the discovery user set

1. Obtain the discovery export (Section 2.2) as CSV/JSON.
2. Run the assessment tool to profile it and flag missing attributes:

```
python discovery-profile.py --input discovery-users.csv --report Discovery-Profile-Enterprise-Assess.js
```

Optionally add `--google-directory` to enrich/confirm against the live Google directory (read-only, Admin SDK Directory scope).

1. **Artifact:** `Discovery Profile – Enterprise – Assess` (the JSON report). It lists user count, data

volume distribution, delegate and shared-mailbox clusters, and any rows missing the attributes the batcher needs.

A2. Confirm the throttling / API-limit model

1. In **Google Cloud console** → **APIs & Services** → **Quotas**, note the current quotas for the APIs your

automation and engine use.

1. Review the current **Microsoft Graph** throttling guidance for the resources your engine reads, and

your engine's documented concurrency guidance.

1. Record these as the input file for the throttle planner. **Confirm current numbers — they change.**

Gate A (Assess → Design): the discovery profile has no blocking missing-attribute rows for the pilot population, and the current limit numbers for both vendors are recorded. If not, remediate the export or gather the limits before proceeding.

Stage B — Design (batch the waves, budget the throttle)

B1. Build the wave batches

1. Run the wave batcher against the profiled set:

```
python wave-batcher.py --input discovery-users.csv --max-users 500 --pilot-size 25 \
    --keep-clusters --out waves.csv --report Wave-Plan-Enterprise-Design.json
```

- `--max-users` sizes each wave; `--pilot-size` carves the first, smallest **pilot** wave.
- `--keep-clusters` keeps delegate groups and shared-mailbox members in the **same** wave.

1. **Artifacts:** one named wave per batch, e.g. `Migration Wave – Pilot – Pilot`,

`Migration Wave – Finance – Wave 2`, `Migration Wave – Sales – Wave 3`. The pilot wave is always **first** and small.

B2. Compute the throttle budget per wave

1. For each wave, compute concurrency + pacing from the recorded limits:

```
python throttle-planner.py --waves waves.csv --limits limits.json \
    --window-hours 8 --report Throttle-Budget-Enterprise-Design.json
```

1. The planner outputs, per wave, a **recommended concurrent-worker count and pacing** that keeps the

projected request rate under both the Microsoft Graph and Google quota ceilings for the wave's data volume and window — plus headroom for `Retry-After` backoff.

1. **Artifact:** `Throttle Budget – <Wave> – Design` (one row per wave in the report).

Gate B (Design → Configure): every wave fits its window inside the throttle budget with headroom, delegate/shared-mailbox clusters are intact, and the pilot wave is the smallest and first. If a wave cannot fit its window under the limits, split it, widen the window, or request a quota increase — do **not** raise concurrency past the budget.

Stage C — Configure (apply the budget, stage the artifacts)

C1. Apply the throttle budget to the engine

1. In the engine, set the concurrency/pacing for the wave to the planner's recommended values (per the

engine's own docs — **verify for your version**).

1. Stage the wave's user mapping (source → target) in the engine as the engine expects it (its project/

mapping/CSV format).

C2. Prepare validation and triage artifacts

1. Load the validation criteria (Section 2.5) as the config for the validation tool.
2. Initialize the hypercare tracker for the wave:

```
python hypercare-triage.py init --wave "Migration Wave - Pilot - Pilot" \
    --sla-config sla.json --store hypercare.json
```

1. **Artifacts:** `Validation Criteria – Enterprise – Configure`, `Hypercare Tracker – Pilot – Pilot`.

Gate C (Configure → Pilot): engine concurrency matches the budget, the pilot mapping is staged, validation criteria are loaded, and the hypercare tracker is initialized with the SLA.

Stage D — Pilot (run the pilot wave end to end: pre-stage → delta → cutover)

Run the **three-pass sequence** on `Migration Wave – Pilot – Pilot`. Each transition has a gate.

D1. Pre-stage pass (no user impact)

1. Start the engine's **pre-stage** (bulk historical copy) for the pilot wave, days ahead of the change window. No mail-flow or DNS change happens here.

1. Monitor throttling: confirm the observed request rate stays within the budget and that `Retry-After` backoff is not sustained (the throttle planner's `--observe` mode can sample engine/API logs if you feed it a rate file).

1. **Gate D1:** pre-stage completes with the engine reporting no unrecoverable errors and throttling within budget. If throttling is sustained, lower concurrency to the budgeted value and re-run — do not push higher.

D2. Delta pass(es) (incremental catch-up)

1. Run one or more **delta** passes to catch items created since pre-stage. No user impact yet.

2. **Gate D2:** the last delta completes cleanly and the residual delta size is small enough to finish inside the cutover window.

D3. Cutover (final delta + mail-routing/DNS switch — change window, high-impact)

1. **Readiness check** — before touching mail flow, confirm the wave is safe to cut over:

```
python cutover-readiness.py --wave "Migration Wave - Pilot - Pilot" \
    --waves waves.csv --checks readiness.json --report Cutover-Readiness-Pilot-Pilot.json
```

It verifies target accounts exist and are licensed, delegates/shared-mailbox members are all in this wave (no split cluster), the pre-stage/delta completed, and coexistence routing is in place.

1. **Go/No-Go gate (Gate D3):** readiness report is all-green and the rollback approver (Section 2.7) authorizes. If any check is red, **No-Go** — remediate and re-check.

1. On **Go**, in the change window: run the engine's **final delta**, then switch **mail routing / DNS** for the wave per the coexistence design (this is the high-impact change).

1. **Rollback (if the gate fails after the switch):** revert the mail-routing/DNS change to the pre-cutover state (restore the prior MX/routing per the change record), leave users on the source, and reschedule. Because pre-stage/delta only *copied* data, reverting routing returns the wave to the coexistence state with no data loss; document the failure and its cause before retrying.

Stage E — Validate (prove the pilot wave is done)

1. Run the post-migration validation against the pilot wave:

```
python validate-migration.py --wave "Migration Wave - Pilot - Pilot" \
    --source-manifest source-counts.csv --criteria validation-criteria.json \
    --report Validation-Report-Pilot-Pilot.json --html Validation-Report-Pilot-Pilot.html
```

It checks, per user and per wave: **item counts** by folder/label vs. source, **permissions/sharing** parity, **mail flow** (test message routes to the target; old address resolves under coexistence), **delegate** access, and **calendar/free-busy**.

1. **Artifact:** `Validation Report – Pilot – Pilot`.
2. **Gate E (Validate → Expand):** the Validation Report passes against the agreed criteria (variances

within tolerance, permissions/mail/delegates/calendar all green). Only a **passing pilot** unlocks expansion. Log any failures into the hypercare tracker and resolve before declaring the pilot done.

Stage F — Expand (roll the proven runbook wave by wave)

1. For each subsequent wave (`Migration Wave – Finance – Wave 2`, …), repeat **C** → **D** → **E** using the

already-computed throttle budget and the proven runbook. Do not change concurrency past the budget.

1. **Hypercare each wave.** During the post-cutover window, log every reported issue:

```
python hypercare-triage.py add --wave "Migration Wave - Finance - Wave 2" \
    --summary "Delegate access missing" --severity S2 --owner ops --store hypercare.json
python hypercare-triage.py report --store hypercare.json --report Hypercare-Report-Finance-Wave2.json
```

The tracker enforces the SLA per severity and reports breaches.

1. **Per-wave gate:** a wave is **not** "done" until its Validation Report passes **and** its hypercare

queue has no open SLA-breaching issues. Only then start the next wave.

1. **Expansion pacing:** run waves at a cadence the hypercare team's SLA can absorb — do not start a new

wave while the previous wave's hypercare queue is breaching.

Artifact register (named, reused everywhere)

Artifact	Created in	Reused in
`Discovery Profile – Enterprise – Assess`	A1	B1, testing
`Migration Wave – <Scope> – <Stage>`	B1	C, D, E, F, testing, triage
`Throttle Budget – <Wave> – Design`	B2	C1, D1
`Validation Criteria – Enterprise – Configure`	C2	E
`Hypercare Tracker – <Wave> – <Stage>`	C2	F
`Cutover Readiness – <Wave> – <Stage>`	D3	Gate D3

Artifact	Created in	Reused in
`Validation Report – <Wave> – <Stage>`	E	Gate E, audit

5. Testing & Validation

Each test below states the procedure and its **expected pass/fail** signal. Run the per-wave tests (T4–T9) against the **pilot wave first** and clear its gate before expanding; then run the same tests on every subsequent wave. Tests use the named artifacts from the Implementation Guide.

T1 — Discovery profile completeness

- **Procedure:** Run `discovery-profile.py --input discovery-users.csv`. Inspect the `Discovery Profile – Enterprise – Assess` report's `missing_attributes` and `clusters` sections.
- **Pass:** every pilot-population row has UPN, department, manager, and delegate/shared-mailbox references; delegate/shared clusters are identified.
- **Fail:** rows missing batching attributes are listed → fix the export before batching (Gate A).

T2 — Cluster integrity in the wave plan

- **Procedure:** Run `wave-batcher.py --keep-clusters …`, then inspect the `Wave Plan` report's `split_clusters` field.
- **Pass:** `split_clusters` is empty — every manager+delegate group and every shared-mailbox membership is contained within a single wave.
- **Fail:** any cluster is split across waves → a delegate would lose access at cutover; re-batch.

T3 — Throttle budget fits the window

- **Procedure:** Run `throttle-planner.py --waves waves.csv --limits limits.json --window-hours 8`. Read the per-wave `fits_window` and `headroom_pct`.
- **Pass:** every wave reports `fits_window: true` with positive `headroom_pct` under **both** the Microsoft Graph and Google quota ceilings.
- **Fail:** a wave reports `fits_window: false` or negative headroom → split the wave, widen the window, or request a quota increase (do **not** raise concurrency past the budget). Gate B.

T4 — Pre-stage throttling stays in budget

• **Procedure:** During the pre-stage pass, sample the engine/API request rate (feed a rate file to `throttle-planner.py --observe`).

• **Pass:** observed rate stays at or below the budgeted rate; no sustained `429` / `RESOURCE_EXHAUSTED` with `Retry-After` backoff.

• **Fail:** sustained throttling → lower concurrency to the budgeted value and re-run (Gate D1).

T5 — Cutover readiness (go/no-go)

• **Procedure:** Run `cutover-readiness.py --wave "<wave>"` …. Read the `Cutover Readiness` report.

• **Pass:** all checks green — target accounts exist and are licensed, no split clusters, pre-stage/delta complete, coexistence routing in place.

• **Fail:** any check red → **No-Go**; remediate and re-run before touching mail flow (Gate D3).

T6 — Item-count parity (post-migration)

• **Procedure:** Run `validate-migration.py --wave "<wave>" --source-manifest source-counts.csv --criteria validation-criteria.json. Read the counts section of the Validation Report`.

• **Pass:** per-user item counts by folder/label match source within the **agreed tolerance** (accounting for documented engine dedup/skips).

• **Fail:** a user's count variance exceeds tolerance → investigate (throttling gap, incomplete delta, or unsupported item type) and re-run the relevant delta. See Troubleshooting.

T7 — Permission & sharing parity

• **Procedure:** In the same `Validation Report`, read the `permissions` section.

• **Pass:** Drive sharing / mailbox permissions on the target match the required source states per the validation criteria.

• **Fail:** a mismatch (e.g. a Shared Drive member missing, a delegate not granted) → remediate the permission and re-validate.

T8 — Mail flow & delegates

• **Procedure:** The `Validation Report` `mailflow` and `delegates` sections; also send a live test message to a migrated user's **old** and **new** addresses.

• **Pass:** test mail reaches the target mailbox; the old address still resolves under coexistence; each configured delegate can open the delegated mailbox/calendar.

• **Fail:** mail bounces/double-delivers, or a delegate is denied → check coexistence routing and delegate grants (Troubleshooting).

T9 — Calendar & free/busy

• **Procedure:** The `Validation Report` `calendar` section; cross-check a free/busy lookup between a migrated user and a not-yet-migrated user.

• **Pass:** calendar events migrated within tolerance; free/busy resolves across the coexistence divide.

• **Fail:** missing events or free/busy gaps → verify calendar interop / delta completeness.

T10 — Hypercare SLA compliance

• **Procedure:** Run `hypercare-triage.py report --store hypercare.json`. Read `sla_breaches`.

• **Pass:** `sla_breaches` is empty for the wave — every logged issue is within its severity SLA.

• **Fail:** one or more issues breach SLA → the wave is **not** done; resolve before starting the next wave (Gate F).

T11 — Safe-by-default tool behavior

• **Procedure:** Run each write-capable tool without `--apply` and re-run the batcher/planner (read-only by nature) on a scratch copy.

• **Pass:** no tool mutates a directory, engine, or mail-routing state without an explicit `--apply`; the assessment, batcher, planner, readiness, and validation tools are read-only.

• **Fail:** any tool writes without `--apply` → stop and fix before using it against production.

6. Troubleshooting

Common failure modes across migration cutover operations, with the symptom, likely cause, and resolution. Symptoms reference the named artifacts and tests from the Implementation and Testing guides.

Throttling & API limits

429 / RESOURCE_EXHAUSTED *storms stall a wave*

• **Symptom:** The engine logs sustained `429 Too Many Requests` (Microsoft Graph) or `RESOURCE_EXHAUSTED` (Google APIs), throughput collapses, and the pre-stage/delta pass misses its window (T4 fails).

• **Likely cause:** Concurrency was set above the `Throttle Budget – <Wave> – Design`, or the limits in

`limits.json` were stale/optimistic.

• **Resolution:** Lower the engine concurrency to the planner's recommended workers/pacing — do **not**

push higher. Confirm `Retry-After` backoff is honored (back off exponentially, don't hammer). Re-confirm current limits in **Google Cloud console** → **APIs & Services** → **Quotas** and against current Microsoft Graph throttling guidance, re-run `throttle-planner.py`, and if a wave genuinely needs more, **request a quota increase** or split the wave / widen the window.

A wave won't fit its window even at the budget

• **Symptom:** `throttle-planner.py` reports `fits_window: false` / negative `headroom_pct` (T3 fails).

• **Cause:** The wave's data volume is too large for the window under the binding vendor limit.

• **Resolution:** Split the wave (`wave-batcher.py --max-users` smaller, or by volume), widen the change

window, or request a quota increase. Never "fix" it by raising concurrency past the budget.

Wave design / clusters

A delegate or shared-mailbox user loses access at cutover

• **Symptom:** After cutover a manager's delegate can't open the mailbox/calendar, or a shared-mailbox

member is locked out; `cutover-readiness.py` flagged `no_split_clusters: red` (T2/T5).

• **Cause:** The dependency cluster was split across two waves — some members migrated, others didn't.

• **Resolution:** Re-batch with `wave-batcher.py --keep-clusters` so the whole cluster is in one wave;

confirm `split_clusters` is empty before accepting the plan. For an already-split cluster, migrate the stragglers into the same wave (or accelerate them) and re-grant the delegate/shared access.

Batcher can only split alphabetically

• **Symptom:** Waves look like arbitrary A–M / N–Z splits; clusters are ignored.

• **Cause:** The discovery export is missing `delegates` / `shared_mailboxes` / `manager` attributes

(T1 flagged `missing_attributes`).

• **Resolution:** Enrich the export from Entra ID / Exchange Online (and optionally

`discovery-profile.py --google-directory` for the target view), then re-batch.

Item counts / delta completeness

Post-migration item counts don't match source

• **Symptom:** `validate-migration.py` reports a user's `variance_pct` over tolerance (T6 fail).

- **Likely causes & resolution:**

- **Legitimate engine dedup/skips** (system items, duplicates) → confirm the behavior in the engine's docs and set the agreed `count_tolerance_pct` in `validation-criteria.json` accordingly. This is a criteria issue, not a data-loss issue.

- **Incomplete delta** — items created after the last delta weren't moved → run another delta pass and re-validate.

- **Throttling gap** — a throttled pass silently skipped items → check the engine error log, re-run the affected pass, re-validate.

- **Unsupported item type** — the engine doesn't migrate a specific type → confirm against the engine's documented supported-items list (**verify for your version**) and handle out-of-band if needed.

Permissions & sharing

Drive sharing or mailbox permissions missing after migration

- **Symptom:** `validate-migration.py permissions` check red (T7): a Shared Drive member or mailbox delegate isn't present on the target.

- **Cause:** The engine migrated content but not all ACLs, or an external/again-not-yet-migrated principal couldn't be mapped.

- **Resolution:** Re-run the engine's permission/ACL pass for the affected objects (per its docs), or apply the missing grant, then re-validate. For principals that aren't migrated yet, note them and re-check after their wave.

Mail flow & coexistence

Mail bounces or double-delivers during coexistence

- **Symptom:** Test mail to a migrated user bounces, or arrives in both the source and target mailbox (T8 fail).

- **Cause:** Mail-routing / dual-delivery for the wave isn't correctly configured, or the DNS/MX cutover is half-applied.

- **Resolution:** Verify the coexistence routing (dual/split delivery) per the program's coexistence design; confirm the wave's MX/routing state matches the intended cutover step. If cutover was mid-switch when it failed, **roll back** the routing/DNS change to the pre-cutover state (returns the wave to coexistence, no data loss) and reschedule.

Old address stops resolving after cutover

- **Symptom:** Mail to a user's pre-migration address is rejected.

- **Cause:** The source alias/routing was removed before coexistence was meant to end.

- **Resolution:** Restore the source-side routing/alias for the coexistence period per the change record.

Calendar / free-busy

Free/busy fails across the migration divide

- **Symptom:** A migrated user can't see a not-yet-migrated user's free/busy (or vice versa) (T9 fail).
- **Cause:** Calendar interop for the coexistence state isn't configured, or events didn't fully migrate.
- **Resolution:** Verify calendar interop per the coexistence design; re-run the calendar delta for the

user if events are missing, then re-validate.

Automation & credentials

Google API calls return 401/403 from the tools

- **Symptom:** `discovery-profile.py --google-directory`, `cutover-readiness.py --verify-google`, or

`validate-migration.py` live pulls fail with 401/403.

- **Cause:** The service account isn't authorized for the read scope, domain-wide delegation isn't set,

or `--delegated-admin` is wrong.

- **Resolution:** In **Admin console** → **Security** → **API controls** → **Domain-wide delegation**, authorize the

service account for exactly the **read-only** scopes each tool's `--help` lists; pass the correct `--delegated-admin`. Use OAuth/ADC — never hardcode or commit keys.

A tool changed something unexpectedly

- **Symptom:** Concern that a tool mutated a directory or routing (T11).
- **Cause:** This should not happen — assessment, batcher, planner, readiness, and validation are

read-only, and every write path is gated behind `--apply`.

- **Resolution:** Re-run without `--apply`; confirm the tool only wrote plan/report files. Report any

tool that writes without `--apply` as a defect and stop using it against production until fixed.

Hypercare

A wave is "done" but users are still blocked

- **Symptom:** `validate-migration.py hypercare report` shows `sla_breaches` for the wave (T10 fail).
- **Cause:** Post-cutover issues were logged but not resolved within their severity SLA.
- **Resolution:** The wave is **not** done — resolve the breaching issues before starting the next wave.

If the same issue recurs across waves, fix the runbook/readiness check so future waves prevent it.

7. Best Practices

Durable practices for running migration cutover operations at enterprise scale, engine-agnostically.

Wave design

- **Pilot first, small, and representative.** The first wave should be small (tens of users) but

representative — include a shared mailbox, a manager+delegate cluster, and a heavy Drive user — so the pilot exercises the real runbook, not just easy cases.

- **Never split a dependency cluster across waves.** A manager and their delegates, and a shared mailbox

and its members, must migrate in the **same** wave. A split cluster means someone loses access at cutover. Always run the cluster-integrity test (T2) before accepting a wave plan.

- **Size by data volume, not just headcount.** Ten users with 100 GB mailboxes are a bigger wave than

fifty with 5 GB. Batch on projected bytes/items so the throttle budget is meaningful.

- **Keep wave names stable.** Use `Migration Wave – <Scope> – <Stage>` and never rename mid-flight —

the name is the join key across the batcher, engine mapping, readiness, validation, and hypercare.

Throttling & API limits

- **Budget before you run; don't discover limits mid-cutover.** Compute concurrency and pacing against

both vendors' *current* limits before the wave starts. A 429 storm at cutover is a self-inflicted incident.

- **Respect `Retry-After`; back off, don't hammer.** When Microsoft Graph or a Google API returns 429 /

`RESOURCE_EXHAUSTED`, honor the `Retry-After` and back off exponentially. Sustained retries are a signal to *lower* concurrency to the budgeted value, not to push harder.

- **Leave headroom.** Budget to a fraction of the ceiling (the planner defaults to headroom) so normal

variance and backoff don't tip a wave into throttling.

- **Re-confirm limits per engagement.** Microsoft Graph throttling and Google quotas change and vary by

license/tenant. Treat any number in this solution as illustrative and **verify current values** — request quota increases where a wave genuinely needs more.

Sequencing (pre-stage → delta → cutover)

- **Pre-stage early, cutover late.** Move the bulk of historical data days ahead with no user impact;

keep only a small final delta for the change window so cutover is short and low-risk.

- **Gate every transition.** Pre-stage → delta → cutover each has an explicit pass/fail gate. Do not let

a wave slide from one pass to the next without meeting the gate.

• **Cutover is a change-controlled event.** Mail-routing/DNS switches are high-impact; make them in an approved window with a named rollback approver and the rollback steps written down *before* you start.

• **Always have a way back.** Because pre-stage/delta only copy data, reverting mail routing returns a wave to coexistence with no data loss. Never cut over a wave whose rollback you haven't confirmed.

Post-migration validation

• **Validate every wave; "done" means evidenced.** A wave isn't done because the engine said "complete"
— it's done when the Validation Report passes against the agreed criteria. Keep the reports as the audit trail.

• **Agree tolerances up front.** Some engines legitimately deduplicate or skip system items, so counts won't match to the item. Define acceptable variance *before* the pilot so every wave is judged the same way.

• **Check the things users feel first.** Mail flow, delegates, and calendar/free-busy are what a user notices on day one — validate them explicitly, not just raw item counts.

Hypercare

• **Run hypercare against an SLA, not vibes.** Log every post-cutover issue with severity, owner, and SLA; a wave with SLA-breaching issues is not finished.

• **Pace expansion to hypercare capacity.** Don't start a new wave while the previous wave's hypercare queue is breaching — the team can't support two tails at once.

• **Feed hypercare patterns back into the runbook.** If the same issue recurs across waves (a delegate grant that's always missed), fix the runbook/readiness check, not just the ticket.

Automation & safety

• **Safe by default.** Assessment, batching, planning, readiness, and validation are read-only; any write is gated behind `--apply`. Prove tools are read-only (T11) before running against production.

• **Least-privilege, no secrets in code.** Use OAuth / ADC or a domain-wide-delegated service account with only the **read** scopes the tools need. Never hardcode keys or commit exported key files.

• **Keep the automation engine-agnostic.** Drive engines through their documented API/CSV/mapping interfaces; do not couple the orchestration to one vendor's private internals so the toolkit outlives any single engine choice.

8. Deliverables Summary

Everything included in this solution, on one page.

Documentation

• **Solution Overview** — the tool-agnostic cutover-operations model: wave batching, throttling/API-limit

management, pre-stage→delta→cutover sequencing, post-migration validation, and hypercare; and how this solution positions against the program-plan and per-service/tool migration solutions.

• **Requirements Guide** — solution-specific prerequisites (chosen/connected engine, discovery user set,

both-vendor limit model, read-only service identity, validation criteria, hypercare team + SLA, change window + rollback authority), building on the shared Google category requirements.

• **Implementation Guide** — exact ordered steps through **Assess → Design → Configure → Pilot →

Validate → Expand**, pilot-first, with every artifact named (`Migration Wave – <Scope> – <Stage>`), explicit pass/fail gates, and a documented cutover rollback.

• **Testing & Validation** — 11 tests (T1–T11) each with an expected pass/fail signal, covering discovery

completeness, cluster integrity, throttle-budget fit, pre-stage throttling, cutover readiness, item-count/permission/mail/delegate/calendar parity, hypercare SLA, and safe-by-default tool behavior.

• **Troubleshooting** — common failure modes and resolutions: throttling / API-limit 429s, permission

errors, incomplete-item / delta issues, split-cluster cutover breakage, mail-flow/coexistence problems, and hypercare SLA breaches.

• **Best Practices** — durable practices for wave design, throttling, sequencing, validation, hypercare,

and safe automation.

• **Executive Summary** — outcome, approach, and why the solution is low-risk.

The 5 Python tools (`scripts/`)

Google Workspace Admin SDK / API automation (with Microsoft Graph read where the source must be read). Each is **read-only or** `--dry-run` **by default**, uses OAuth / ADC, and carries **no secrets in code**.

1. `discovery-profile.py` — *Assessment / discovery (read-only).* Profiles the discovery user-set

export (and optionally the live Google directory via Admin SDK Directory) to report user count, data distribution, delegate and shared-mailbox **clusters**, and any rows missing attributes the batcher needs. Produces `Discovery Profile – Enterprise – Assess`.

1. `wave-batcher.py` — *Orchestration helper (read-only planning; writes only the plan files).*

Batches the user set into ordered, size-limited, **dependency-aware** waves (keeps delegate and shared-mailbox clusters together), carving a small pilot wave first. Emits `waves.csv` and the `Migration Wave – <Scope> – <Stage>` plan; flags any split clusters.

1. `throttle-planner.py` — *Policy / throttling / API-limit check (read-only).* Computes a per-wave

throttle budget (concurrent workers + pacing) that keeps projected request rate under both the Microsoft Graph throttling model and the Google API quotas for the wave's volume and window, with headroom; `--observe` mode checks a sampled rate file against budget. Produces `Throttle Budget – <Wave>`

– Design.

1. cutover-readiness.py — *Cutover-readiness / config-scope helper (read-only).* Runs the pre-cutover

go/no-go checks for a wave — target accounts exist & licensed, no split clusters, pre-stage/delta complete, coexistence routing in place — and emits an all-green/red Cutover Readiness – <Wave> – <Stage> report that gates the mail-flow switch.

1. validate-migration.py — *Validation / report generator (read-only).* Post-migration validation:

compares per-user **item counts** (vs. a source manifest), **permissions/sharing**, **mail flow**, **delegates**, and **calendar/free-busy** against the agreed criteria; emits the JSON + HTML Validation Report – <Wave> – <Stage> — the evidence a wave is done. *(A companion hypercare-triage.py tracker records post-cutover issues against the SLA; it is bundled with the validation tool as the evidence/close-out pair.)*

> *Safe-by-default: every write path is gated behind an explicit --apply; assessment, batching, planning, readiness, and validation are read-only. Credentials are supplied via OAuth / Application Default Credentials or a least-privilege domain-wide-delegated service account — never hardcoded.* pip install google-api-python-client google-auth *where the Google APIs are used.*

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

cutover-readiness.py

discovery-profile.py

throttle-planner.py

validate-migration.py

wave-batcher.py